

Tetris-99++: MARL Evolutionary Learning via PSRO & SESIL Application

Stephen Mamosian*

Carleton University

Ottawa, Canada

StephenMamosian@cmail.carleton.ca

ABSTRACT

Tetris-99++ is a complex and interesting spin-off of legacy Tetris. It introduces Tetris in a multiplayer setting and a mechanic that allows interaction of two Tetris boards playing in parallel. The mechanic that makes Tetris-99++ so mind-boggling to look at is having control over another player's board's next object. This level of control of the game can create cooperation and competition within the same run of the environment. Understanding the behaviour of the agents becomes so intriguing. A combination of two complex algorithms, PSRO and SESIL, is used to simulate evolution-like improvement within a population of agents similar to beings in nature. Having a rather difficult environment like Tetris-99++ challenges these complex algorithms to see if they can benefit from their mutation and offspring features to evolve through a learning curve that is less steep than using plain Independent Q-Learning models. To do so, the new custom environment is required to be standardised and packaged in a standard library like PettingZoo to allow leveraging of models and algorithms that support the PettingZoo library. RLLib and EasyMarl are two libraries that provide models and algorithms like DQN, Independent Q-Learning, PSRO, and SESIL to use against our standard custom PettingZoo environment, Tetris-99++. Matching the behaviour of the agents within the environment to their immediate return displays a good observation of agents' thinking mindset within different models. There are lots of configurations and changes that can be made to this new custom environment to view the effect on learning and outcomes within different models and algorithms, such that it opens up many doors to new Multi-Agent Reinforcement Learning projects and research.

ACM Reference Format:

Stephen Mamosian. 2026. Tetris-99++: MARL Evolutionary Learning via PSRO & SESIL Application. In *Proc. of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2026)*, Paphos, Cyprus, May 25 – 29, 2026, IFAAMAS, 5 pages.

1 INTRODUCTION

Tetris-99++ is a very complex and intriguing multi-agent environment that requires very smart solutions to solve. This report develops and standardises a complex multi-agent environment, then evaluates whether learning algorithms can learn effective behaviour within it. Learning is assessed through training results that show whether an agent can consistently outperform the return

bounds of a random strategy. PSRO and SESIL provide complementary approaches to multi-agent learning: PSRO addresses strategic interaction by maintaining and expanding sets of policies, while SESIL uses social and evolutionary mechanisms to combine useful behaviours across a population [6, 7]. This project investigates an innovative combination of these approaches in a challenging environment where both strategic interaction and learned behavioural diversity matter. Tetris-99++ provides such a test-bed: it extends Tetris-99 with a custom two-player mechanic in which an agent can influence the opposing player's next block. The environment therefore provides both individual decision-making and interaction between agents. To make this investigation possible, the environment must first be implemented in a standardized, reusable interface and then validated as learnable. The project consequently follows three stages: developing an appropriately complex environment based on Tetris-99; packaging it through a standard interface so it can be reused by existing learning libraries; and validating that it is in principle solvable by reinforcement-learning algorithms. The experiments first test single-agent Tetris-99++ with DQN and then test its multiplayer mechanics with Independent Q-Learning. The main contributions of this work are: (1) the development of the Tetris-99++ environment and its standardization through the PettingZoo interface; (2) the integration and evaluation of reinforcement-learning baselines, including DQN and Independent Q-Learning; and (3) the implementation and release of the Tetris-99++ environment as an open-source project in the accompanying GitHub repository [4].

2 BACKGROUND

2.1 Tetris

This project introduces a new environment called Tetris-99++. It is called so due to the fact that the base mechanics of the game follow the legacy Tetris game; however, more mechanics are introduced to create unique and very interesting strategies. Legacy Tetris is a video game that was developed by a Soviet software engineer named Alexey Pajitnov in June 1984. This introduces a 20-block-high and 10-block-wide board that drops seven different types of objects from the top down to the bottom. These include the line, box, L shape, inverted L shape, Z shape, S shape, and T shape. Objects spawn one at a time at the top of the board and float down to the bottom until they are set onto another settled object. When an object is settled/dropped, a new object spawns at the top of the board. This cycle continues until the board is overfilled and any part of any object that has been settled exceeds the first 3 rows of the board. In such a case, the game is over. To avoid this condition, the player can maneuver the objects floating down the

*Student ID: 101224394

board by moving them in the x or y axis while also allowing 90-degree rotations. When a player fills out a row, meaning in a row all 10 pixels/blocks are filled, the row is cleared, and all rows above are moved down appropriately. This mechanic allows a good player to stay alive longer within the game.

Tetris-99++ uses the base mechanics of Tetris 99, a competitive multiplayer version of Tetris in which players guide falling Tetriminos, clear horizontal lines, and can send garbage to opponents [9]. Tetris-99++ changes the control mechanism: rather than continuously moving a falling block through time, the action space contains the possible settling positions and rotations for the current block. At each tick, the player therefore chooses a column and rotation at which to settle the block. This reduces the decision space without necessarily making the underlying game simpler. Tetris-99++ also changes the interaction model. In Tetris 99, interaction is primarily through attacks and targeting, whereas in Tetris-99++ an agent directly chooses the opposing player's next block. With two agents playing simultaneously, this creates both competitive and potentially cooperative behaviour because the continuation of one player's game affects the other player's outcome.

2.2 Learning Algorithms

A couple of models are used in this report to validate the solvability of Tetris-99++. One of those is DQN or Independent Q-Learning. Independent Q-Learning is a commonly used multi-agent reinforcement-learning baseline in which each agent learns independently while treating the other agents as part of its environment [6]. Independent Q-Learning is a DQN model (Deep Q-Network) that runs independently for each agent and does not interact with other players' models at all. DQN itself is a simple model that assigns estimated Q-values per action-state pairs and updates them during training using the Bellman equation [8].

The other more complex algorithm is PSRO (Policy-Space Response Oracles) [6]. PSRO maintains a population of policies for each player and evaluates the policies against one another to construct an empirical payoff table. A meta-solver then determines a distribution over the policies in each population. An oracle, such as Independent Q-Learning, is used to train a new policy as an approximate best response to the opponent's policy distribution. The newly trained policy is added to the corresponding population, expanding the set of available strategies. This process is repeated so that the empirical game incorporates increasingly diverse responses to the strategies discovered so far. The benefit of PSRO is therefore not stated as universally outperforming Independent Q-Learning; rather, it provides a framework for addressing the policy-overfitting and generalization issues that can arise when independent learners train against a limited set of opponents [6].

SESL (Social, Evolutionary Supported Learning) introduces a population-level mechanism for social learning through evolution [7]. The central idea is that an individual agent may observe useful behaviour in another agent but cannot directly absorb that behaviour. Instead, agents use observed performance to select compatible mates, and a genetic-merger operation produces children that combine aspects of the parents' learned behaviour. SESL therefore treats evolution as a population process rather than as a property

of an individual agent. In this project, each agent has learned policies represented by neural-network parameters, while high-level descriptions of those learned behaviours are treated as attributes. During a round-robin evaluation, the performance of these behaviours is recorded in a fitness vector. Agents can then use these fitness values to identify complementary behaviours and generate offspring. This provides a formal basis for studying whether social and evolutionary combination can complement the strategic policy expansion performed by PSRO.

3 MODEL + TEST-BED

The model progression in this project is organized around increasing strategic and evolutionary structure rather than assuming in advance that one algorithm is universally more efficient than another. Independent Q-Learning provides a baseline in which each agent learns its own policy. PSRO then maintains and expands policy populations through approximate best responses, while SESL introduces social and evolutionary combination of learned behaviours [6, 7]. The final objective is to investigate whether combining these mechanisms provides useful learning behaviour in Tetris-99++.

The motivation for combining PSRO and SESL is that the two mechanisms address different aspects of population-based learning. PSRO expands the set of policies by training responses to distributions of existing strategies, while SESL allows agents to benefit indirectly from one another through evolutionary combination. In the Tetris-99++ setting, the corresponding objective is to learn behaviours that allow agents to survive for longer while responding to the behaviour of other agents.

Independent Q-Learning learns a policy for each agent without explicitly maintaining a population of alternative policies. PSRO instead maintains an expanding set of policies and repeatedly trains new policies as approximate best responses to distributions over existing policies [6]. SESL operates at the population level: agents evaluate learned behaviours, use the resulting fitness information to identify complementary behaviours, and produce offspring through a genetic-merger operation [7]. Thus, the terms are kept distinct: "parameters" refer to the neural-network weights that encode a learned behaviour, while "attributes" refer to high-level properties used to describe and classify those behaviours.

In Tetris-99++, an agent's parameters are the neural-network weights that encode its learned policy, while attributes are high-level descriptions of the resulting behaviour. During a PSRO iteration, a new policy is trained against a distribution over the opponent's existing policies until it provides an adequate response, after which the policy is added to the population [6]. SESL can then evaluate the resulting behaviours in a round-robin tournament and use the fitness information to identify complementary behaviours for evolutionary combination [7]. The experiments therefore evaluate a sequence of increasingly structured approaches: single-agent DQN, two-agent Independent Q-Learning, and the proposed PSRO/SESL direction. Each stage serves as a test-bed for the next rather than being described as a separate "layer."

4 EXPERIMENTS

To allow the custom-made Tetris environment to be used universally by different models, it is necessary to wrap the new environment into some standard library that the model's library supports. PSRO is a heavy and complex algorithm; therefore, to not reinvent the wheel in this project, first it was attempted to use PSRO out of the box from a MultiAgent Reinforcement Learning library. The first library that was attempted was named MALib [1]. This library, developed in 2021, provided PSRO out of the box. However, after long attempts to integrate Tetris-99++ into MALib due to the lack of documentation, the interest shifted towards OpenSpiel. OpenSpiel provides its own custom environment structure, which is documented better than MALib. After integrating Tetris-99++ into a custom OpenSpiel environment, an attempt to run PSRO was made. However, the runtime was unexpectedly high and the resulting behaviour did not align with the expected learning process. Unfortunately, OpenSpiel also provides little to no documentation about the PSRO algorithm and how it is implemented. After taking a look at the PSRO implementation itself, it appeared to be specialized for decision-tree-like games, meaning it did not work well with Tetris-99++ as game lengths are rather unlimited and a single episode's worth of decision tree in Tetris-99++ is very resource-heavy and doesn't comply well with this type of environment. As a result, with the help of a partner (Tianshu Zhao), an implementation of PSRO was developed from scratch into a new library called EasyMarl [3].

EasyMarl, however, uses the PettingZoo library as its standard for environments. Therefore, Tetris-99++ was rewrapped into PettingZoo custom environment APIs to allow hand-in-hand work with EasyMarl's models. EasyMarl provides the PSRO algorithm; however, it requires an Oracle model. Therefore, before applying the more complex population-based methods, the standardized custom PettingZoo environment was first validated with a simple baseline such as Independent Q-Learning. Since Tetris-99++ was implemented from scratch as a part of this project, a lot of features were implemented besides the core action environment in this project [4], including the board representation, block generation and placement, line clearing, game-over handling, and the two-player mechanism for selecting the opposing player's next block. For the sake of sanity testing and ensuring the new custom environment was learnable and solvable, a standard reinforcement learning model library was found that works well with PettingZoo environments. This library is called RLLib [5]. RLLib provides DQN for single-agent applications and Independent Q-Learning for multiagent applications out of the box. Since Tetris-99++ is developed from scratch, including Tetris itself, it is best to test single-player mechanics and solvability via RLLib DQN and then move over to the multiagent (Tetris-99++ style).

In this project, all training results are compared to the mean reward of running the environment with a random policy and display an upper and lower bound determined from standard deviation to define a zone that, if the algorithm exceeds, is considered as learning/solved. Meaning that the agent was able to find a pattern and understand the game to outperform a random policy in a consistent manner. Below is the result for DQN running Tetris-99++ single player (Figure 1).

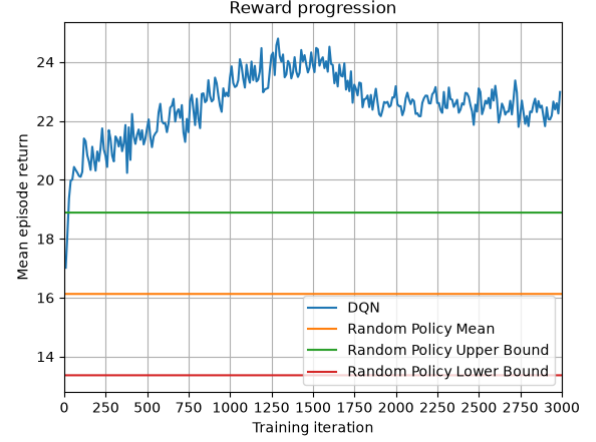


Figure 1: DQN training rewards on single-player Tetris-99++ compared to the mean reward and standard-deviation bounds of a random policy.

After confirming the learning of single player mode of Tetris-99++ (similar to legacy Tetris), the attempt to solve the multiplayer side of Tetris-99++ started via Independent Q-Learning. Below is the result for Independent Q-Learning running Tetris-99++ with two agents (Figure 2).

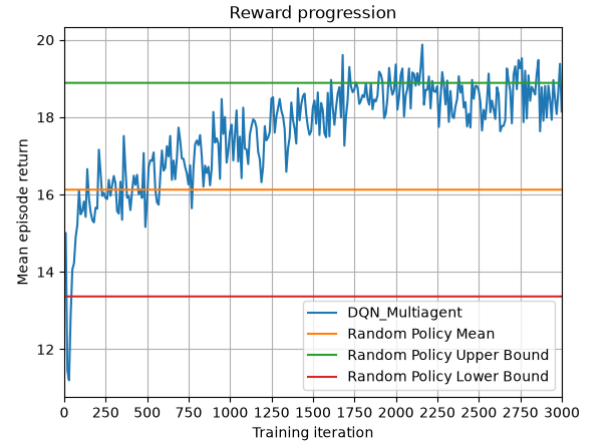


Figure 2: Independent Q-Learning training rewards on two-player Tetris-99++ compared to the mean reward and standard-deviation bounds of a random policy.

Unfortunately, due to the lack of time, the experimentation for Tetris-99++ was stopped here. Further analysis of the figures can be found in the Analysis section, and future work in the Conclusions/Future work section.

5 ANALYSIS

In this section, a deeper analysis is done on the experimentation done for this project and an attempt to understand the results, plots, and graphs. In Figure 1, a clear immediate ascent is visible above the upper boundary of the random policy. Meaning, in the single-agent mode of Tetris-99++, DQN was able to find quick patterns and learn simple tricks to allow its rewards/lifespan to be longer than a random policy. This shows learning and solvability of the environment. However, in the immediate aftermath around training iteration 1,250, there can be seen a stop to the rapid learning and entering a slump. It seems that even through 1,250 learning iterations (each learning iteration consists of many episodes) the agent struggles to find the main mechanic of the game. And that is filling in full rows to clear existing blocks.

In this project, to allow free room to the agent, the observation space that was fed into the model is the board. The board was flattened and sent as a ones-and-zeros array, with ones meaning that the cell had a block within and zeros meaning that the cell was empty. This observation/state is quite difficult to understand by a simple model like DQN. The current figure reports reward rather than lifespan directly, so a separate lifespan-over-training plot would be a useful additional diagnostic for future evaluation. However, as we see in Figure 1, it does find a way to extend its lifespan by spreading the objects/blocks evenly across the board until the board fills up. And learning how to spread evenly can be seen from training iteration 0 to 1,250. From 1,250 onwards, the agent struggles to find another solution and is getting punished for exploring; that is why a descent is visible post 1,250. If the model were trained for possibly more training iterations, there could be a chance that exploring finally pays off and clearing rows is discovered. However, the 3,000 training iterations already take more than 24 hours to train, so only a comparatively small portion of the available project time could be devoted to exploring that later stage of learning.

Given the comparatively limited time available for unrestricted experimentation after the substantial implementation and integration work, focus moved from further refinement of the single-agent model to two-player Tetris-99++ with Independent Q-Learning. In Figure 2, learning similar to Figure 1 can be seen. However, the ascent is very slow and not as drastic as in Figure 1. In the two-player Tetris-99++ environment, a new mechanic exists which does not exist in single-player Tetris-99++. And that is choosing opponents' next block. This extra piece of decision-making, on top of the opponent possibly sabotaging the current agent's next block, makes learning the core mechanics of Tetris even more difficult. However, it is still visible in Figure 2 that climbing through the randomness zone occurs and settles just at/above the upper bound of randomness. Meaning the two Independent Q-Learning agents are not completely random; however, they still struggle to have a very good understanding of Tetris-99++. On the bright side, in the figure below, there is hope in the fact that the agents are learning Tetris-99++'s unique mechanics (Figure 3).

Figure 3 describes the overall distribution of obstacle/block/object types selected for the opposing user. The figure shows a higher frequency for the box obstacle, suggesting that the agents may have developed a preference rather than selecting blocks uniformly at

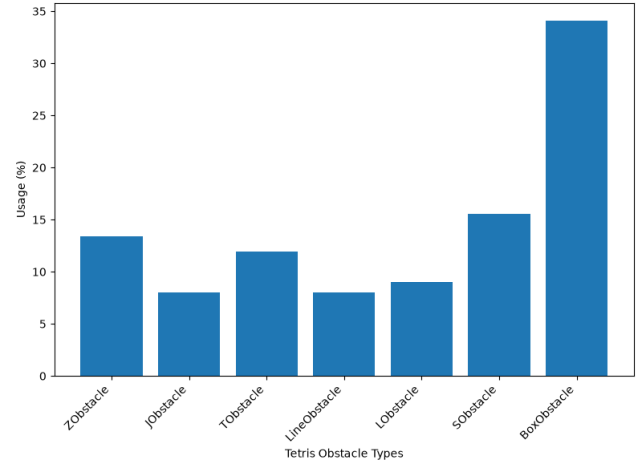


Figure 3: Distribution of the obstacle/block/object types picked for the opposing player over training iterations.

random. Because the current figure aggregates the observations instead of showing how the frequencies change over training, a future version should plot the frequency of each block type as a function of training iteration to directly demonstrate whether this preference emerges over time. One thing that is important to note here is that when an agent reaches a game-over state in Tetris-99++ two players, the opposing agent's state also reaches game-over. This means that if agents help each other by providing the simplest blocks to one another, the overall length of the game would increase, resulting in higher reward/lifespan. Box objects are really easy to interpret and understand by the less competent agents. Because Box objects do not rotate, cutting down their complexity and mechanism to only defining their placement on the x-axis. This observation identifies an additional behavioural attribute that can be studied in future experiments. In particular, the relationship between block-selection preferences and game longevity provides a useful direction for further analysis.

6 CONCLUSIONS/FUTURE WORK

All in all, within this project, a new custom environment has been developed. This complex and interesting environment opens up lots of applications of model research due to its intriguing uniqueness and complexity. Tetris-99++, through some struggle, eventually standardised into the PettingZoo library to allow universal use of the environment. RLLib, which is a standardized library that uses PettingZoo environments, allows running DQN and Independent Q-Learning for Tetris-99++ singleplayer and multiplayer, respectively. Analyzing the results shows the complexity of the environment is intriguing and controllable yet not easy. In Independent Q-Learning analysis, observations on how agents deal with selecting their opponents' blocks, and what their thinking processes might look like and how it benefits them and affects their immediate reward/payoff.

The innovative and complex nature of the central idea, together with the difficulty of integrating population-based algorithms into a new environment, limited the portion of the project in which

unrestricted comparative experimentation could proceed. Common libraries used by other labs can also rely on internal implementation knowledge rather than extensive documentation, which raises the entrance barrier for novel environments. Despite these challenges, the environment was implemented, standardized, and validated with single- and multi-agent baselines. A natural next step is to study how PSRO and a combination of PSRO and SESIL perform in Tetris-99+, including their learning curves and behavioural diversity. Other future work that was a part of the imagination of this project are manipulating the observation of the agents. The current implementation of Tetris-99+ multiplayer provides all agents current boards in flat ones and zeros. Further study on how manipulation and providing more or less or preprocessed state can affect the performance of learning Tetris-99+. Also, the action space of Tetris-99+ can be changed back to legacy Tetris floating blocks with directional manoeuvres like the current implementation of picking settle positions. This can add more complexity and makes Tetris-99+ even more interesting. Allowing floating movements can create object placements and positions that cannot exist in the current implementation of Tetris-99+. This can introduce a “tucking mechanic” allowing a piece of the object to be tucked into a gap to avoid increasing the height of the board. Furthermore, this project opens many doors to new research and projects.

REFERENCES

- [1] Ming Zhou, Ziyu Wan, Hanjing Wang, Muning Wen, Runzhe Wu, Ying Wen, Yaodong Yang, Weinan Zhang, and Jun Wang. 2021. MALib: A Parallel Framework for Population-based Multi-agent Reinforcement Learning. Preprint.
- [2] Marc Lanctot, Edward Lockhart, Jean-Baptiste Lespiau, Vinicius Zambaldi, Satyaki Upadhyay, Julien Pérolat, Sriram Srinivasan, Finbarr Timbers, Karl Tuyls, Shayegan Omidshafiei, Daniel Hennes, Dustin Morrill, Paul Muller, Timo Ewalds, Ryan Faulkner, János Kramár, Bart De Vylder, Brennan Saeta, James Bradbury, David Ding, Sebastian Borgeaud, Matthew Lai, Julian Schrittwieser, Thomas Anthony, Edward Hughes, Ivo Danihelka, and Jonah Ryan-Davis. 2019. OpenSpiel: A Framework for Reinforcement Learning in Games. *CoRR* abs/1908.09453 (2019). arXiv:1908.09453 <http://arxiv.org/abs/1908.09453>
- [3] Tianshu Zhao and Stephen Mamosian. EasyMarl: A Lightweight Multi-Agent Reinforcement Learning Library.
- [4] Stephen Mamosian. Tetris-MARL. <https://github.com/TheSlenk/tetris-marl>
- [5] Zhanghao Wu, Eric Liang, Michael Luo, Sven Mika, Joseph E. Gonzalez, and Ion Stoica. 2021. RLlib Flow: Distributed Reinforcement Learning is a Dataflow Problem. In *Conference on Neural Information Processing Systems (NeurIPS)*. <https://proceedings.neurips.cc/paper/2021/file/2bce32ed409f5ebcee2a7b417ad9beed-Paper.pdf>. See also: Eric Liang, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph E. Gonzalez, Michael I. Jordan, and Ion Stoica. 2018. RLlib: Abstractions for Distributed Reinforcement Learning. In *International Conference on Machine Learning (ICML)*. <https://arxiv.org/pdf/1712.09381>
- [6] Ariyan Bighashdel, Yongzhao Wang, Stephen McAleer, Rahul Savani, and Frans A. Oliehoek. 2024. Policy Space Response Oracles: A Survey. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI 2024)*, 7951–7961. <https://arxiv.org/abs/2403.02227>
- [7] Tianshu Zhao and Zinovi Rabinovich. 2026. SESIL: Social, Evolutionary Supported Learning. In *Proceedings of the 25th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2026)*, 2160–2168.
- [8] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 529–533.
- [9] Nintendo. 2019. Tetris 99. Nintendo Switch.